

未使用功能与优化指南

本文档列出了项目中已实现但未调用的功能，以及预留接口但未完成实现的功能。
目的是为后续开发者提供清晰的优化方向和可尝试的功能点。

一、已实现但未调用的函数

1.1 参数估计模块 (src/parameter_estimation.py)

期货时间序列的MLE估计

函数：

- `log_likelihood_futures(q_matrix, state_sequence, time_step)`
- `log_likelihood_futures_rs(q_matrix, state_sequence, time_step, n_price_states, initial_regime=1)`

功能：

- 实现了基于期货价格时间序列的对数似然函数
- 支持标准CEV模型和CEV-RS机制切换模型
- CEV-RS版本使用前向滤波法处理隐含机制状态

状态： ☒ 已完整实现， ☐ 未被调用

使用场景：

- 当希望从期货历史数据估计模型参数（而非使用期权市场数据校准）
- 适合用于初始化参数，或对比期权校准结果

如何使用：

```
# 1. 将期货价格数据离散化为状态序列
indices, bins = discretize_data(futures_prices, num_bins, x_lower, x_upper)
state_sequence = indices

# 2. 构造Q矩阵
q_matrix = construct_theoretical_q_matrix(bins_centers, initial_price, params,
model_type)

# 3. 计算对数似然
log_likelihood = log_likelihood_futures(q_matrix, state_sequence, time_step)

# 4. 优化（需要实现优化包装函数）
from scipy.optimize import minimize
result = minimize(lambda p: log_likelihood_futures(...), initial_guess,
bounds=...)
```

注意事项：

- 对于CEV-RS模型，机制转换参数 ($\lambda_{12}, \lambda_{21}$) 较难从期货数据识别
- 建议使用期货MLE估计扩散参数 (σ, β)，再用期权数据校准其他参数

机制分析工具

函数:

- `calculate_stationary_distribution(lambda12, lambda21)`
- `calculate_expected_regime_duration(lambda12, lambda21)`
- `analyze_regime_parameters(params)`

功能:

- 计算CEV-RS模型的机制平稳分布
- 计算每个机制的期望持续时间
- 提供参数的经济含义解释

状态: ☒ 已完整实现, ☐ 未被调用

使用场景:

- 在参数估计完成后, 理解机制切换特征
- 分析市场是"平静期"还是"压力期"占主导
- 评估杠杆效应类型 (正向杠杆 vs 反向杠杆)

示例:

```
from src.parameter_estimation import analyze_regime_parameters

# 假设校准得到的参数
params = [0.16, 1.0, 0.16, -1.0, 1.0, 7.0] # sigma1, beta1, sigma2, beta2, lambda12, lambda21

analysis = analyze_regime_parameters(params)

print(f"机制1 (平静期): sigma={analysis['regime1_volatility']:.4f}, beta={analysis['regime1_elasticity']:.4f}")
print(f"机制2 (压力期): sigma={analysis['regime2_volatility']:.4f}, beta={analysis['regime2_elasticity']:.4f}")
print(f"平稳分布:  $\pi_1$ ={analysis['stationary_prob_regime1']:.4f},  $\pi_2$ ={analysis['stationary_prob_regime2']:.4f}")
print(f"期望持续时间:  $E[T_1]$ ={analysis['expected_duration_regime1']:.4f}年,  $E[T_2]$ ={analysis['expected_duration_regime2']:.4f}年")
print(f"主导机制: {analysis['dominant_regime']}, 杠杆类型: {analysis['dominant_leverage_type']}")
```

输出解读:

- `beta > 0`: 反向杠杆 (价格下跌时波动率上升, 平静市场特征)
- `beta < 0`: 正常杠杆 (价格下跌时波动率下降, 压力市场特征)
- 平稳分布接近0.5/0.5表示两个机制频繁切换
- 期望持续时间短 (如<0.5年) 表示市场状态变化快

RAE报告辅助函数

函数:

- `_report_rae(daily_data, params, ctmc_p, price_fn, label="RAE")`

功能:

- 计算并记录相对绝对误差 (RAE)
- 对应论文Eq. (20): $RAE = (1/N) \times \sum |P_{model} - P_{market}| / P_{market}$

状态: ☒ 已实现, ☒ 未被调用

使用场景:

- 需要批量评估定价性能时
- 生成详细的误差统计报告

1.2 期权定价模块 (src/option_pricing.py)

定价收敛监控

函数:

- `monitor_pricing_convergence(value_matrix, time_step, tolerance=1e-6)`

功能:

- 监控动态规划迭代过程中的收敛情况
- 检测数值不稳定或非物理行为

状态: ☒ 已完整实现, ☒ 未被调用

使用场景:

- 开发新定价功能时调试
- 检测Q矩阵或状态空间设置是否合理
- 验证早期行权边界是否正确

期权价值轮廓分析

函数:

- `analyze_option_value_profile(value_matrix, bins_centers, option_params, model_type)`

功能:

- 分析期权价值随价格的变化规律
- 检测异常的价值行为

状态: ☒ 已完整实现, ☒ 未被调用

使用场景:

- 可视化期权价值曲线
- 验证定价算法的正确性
- 检测深度实值/虚值期权的定价问题

跳跃边界条件（CEV-DEJD专用）

函数：

- `apply_jump_boundary_conditions(value_matrix, bins_centers, option_params, n_time_steps)`

功能：

- 处理跳跃扩散模型的边界条件
- 防止跳跃导致的价值泄漏

状态： ☒ 已完整实现， ☒ 未被调用

使用场景：

- 优化CEV-DEJD模型的数值稳定性
- 处理极端跳跃情况

边界价格插值

函数：

- `interpolate_boundary_price(current_price, bins_centers, option_values, transition_probs)`

功能：

- 对早期行权边界进行线性插值
- 提高边界精度

状态： ☒ 已完整实现， ☒ 未被调用

使用场景：

- 提高CEV-RS早期行权边界的可视化质量
- 生成更平滑的边界曲线

模型价格比较

函数：

- `compare_model_prices(price_cev, price_cev_dejd, price_cev_rs, option_params)`

功能：

- 比较三个模型（CEV、CEV-DEJD、CEV-RS）的定价结果
- 评估模型间的差异

状态： ☒ 已完整实现， ☒ 未被调用

使用场景：

- 模型选择研究
- 分析不同模型的适用场景
- 生成模型对比报告

机制值提取 (CEV-RS专用)

函数:

- `extract_regime_values(value_matrix, bins_centers)`

功能:

- 从2N维价值矩阵中提取两个机制的价值
- 分析机制对期权价值的影响

状态: ☒ 已完整实现, ☒ 未被调用

使用场景:

- 分析不同机制下的期权价值差异
- 评估机制转换的经济影响
- 机制风险溢价分析

1.3 CTMC构造模块(src/ctmc_model.py)

函数:

- `calculate_transition_probability(q_matrix, time_step)` (src/ctmc_model.py:296)
- `get_state_index(price, bins_centers)` (src/ctmc_model.py:341)
- `get_regime_state_index(price, bins_centers, regime)` (src/ctmc_model.py:346)
- `extract_regime_q_matrices(q_matrix_full)` (src/ctmc_model.py:363)

当前状态:

- ☒ 已实现
- ☒ 未被调用

使用场景:

- 转移概率独立计算 (用于验证或可视化)
- CEV-RS状态空间索引管理
- 机制切换分析

1.4 数据处理模块 (utility/data_processor.py)

跳跃特征分析

函数:

- `analyze_jump_characteristics(futures_data, threshold_pct=0.02)`

功能:

- 分析期货价格中的跳跃特征
- 估计跳跃频率、大小、不对称性

状态: ☒ 已完整实现, ☒ 未被调用

使用场景:

- 设置CEV-DEJD模型的初始参数
- 评估数据是否适合跳跃模型
- 跳跃风险分析

示例:

```
from utility.data_processor import analyze_jump_characteristics

futures_prices = np.array([503.24, 508.3, 506.5, ...])
jump_stats = analyze_jump_characteristics(futures_prices)

print(f"潜在跳跃次数: {jump_stats['potential_jumps']}")
print(f"跳跃频率: {jump_stats['jump_frequency']:.2%}")
print(f"向上跳跃: {jump_stats['positive_jumps']}, 向下跳跃: {jump_stats['negative_jumps']}")
print(f"跳跃不对称性: {jump_stats['jump_asymmetry']:.2%} (>50%表示更多向上跳跃)")
print(f"平均向上跳跃: {jump_stats['avg_positive_jump']:.4f}")
print(f"平均向下跳跃: {jump_stats['avg_negative_jump']:.4f}")
```

输出解读:

- 跳跃频率 > 5%: 适合CEV-DEJD模型
- 跳跃不对称性 $\neq$ 50%: 存在方向性偏差
- 结果可用于设置CEV-DEJD的初始参数: λ (跳跃频率)、 p (向上概率)、 η_1/η_2 (衰减率)

机制特征分析

函数:

- `analyze_regime_characteristics(futures_data, window_size=20)`

功能:

- 使用滚动窗口识别高/低波动率机制
- 估计机制转换频率

状态: ☒ 已完整实现, ☐ 未被调用

使用场景:

- 评估数据是否存在机制切换特征
- 设置CEV-RS模型的初始参数
- 机制风险分析

示例:

```
from utility.data_processor import analyze_regime_characteristics

regime_stats = analyze_regime_characteristics(futures_prices)

print(f"高波动期占比: {regime_stats['high_vol_proportion']:.2%}")
print(f"高波动期平均波动: {regime_stats['avg_high_vol']:.2%}")
print(f"低波动期平均波动: {regime_stats['avg_low_vol']:.2%}")
print(f"波动比率: {regime_stats['vol_ratio']:.2f} (>1.5表示有明显机制差异)")
print(f"估计转换率:  $\lambda_{12}$ ={regime_stats['estimated_lambda12']:.2f},  $\lambda_{21}$ ={regime_stats['estimated_lambda21']:.2f}")
```

输出解读:

- 高波动期占比 $\approx$ 50%: 机制切换较均衡
- 波动比率 > 1.5: 机制差异明显, CEV-RS适合

- 估计转换率可用于CEV-RS初始参数 (λ_{12} , λ_{21})

杠杆效应分析

函数:

- `analyze_leverage_effect(futures_data, window_size=20)`

功能:

- 分析收益率与波动率变化的相关性
- 判断杠杆效应类型

状态: ☒ 已完整实现, ☒ 未被调用

使用场景:

- 设置CEV/CEV-RS模型的 β (弹性系数) 初始值
- 理解市场杠杆特征

示例:

```
from utility.data_processor import analyze_leverage_effect

leverage_analysis = analyze_leverage_effect(futures_prices)

print(f"相关性: {leverage_analysis['correlation']:.4f}")
print(f"杠杆类型: {leverage_analysis['leverage_type']}")
print(f"强度: {leverage_analysis['strength']:.2f}")
print(f"解读: {leverage_analysis['interpretation']}")

# 输出示例:
# 相关性: 0.25
# 杠杆类型: inverse (反向杠杆)
# 强度: 0.25
# 解读: weak
```

输出解读:

- 相关性 > 0: 反向杠杆 (价格↑时波动率↑), 平静市场特征
- 相关性 < 0: 正常杠杆 (价格↓时波动率↑), 压力市场特征
- 强度 > 0.3: 强杠杆效应
- 相关性可设置 β 初始值: $\beta \approx \text{相关性} \times 5$

状态空间参数建议

函数:

- `suggest_state_space_params(futures_data, model_type='cev')`

功能:

- 根据数据特征建议最优的状态空间参数
- 动态调整扩展比例和网格数量

状态: ☒ 已完整实现, ☒ 未被调用

使用场景:

- 提高定价效率（减少不必要的网格点）
- 增加数值稳定性（避免边界截断）
- 自适应优化

示例：

```
from utility.data_processor import suggest_state_space_params

suggestions = suggest_state_space_params(futures_prices, model_type='cev_dejd')

print(f"建议上界扩展: {suggestions['delta_upper']:.2%}")
print(f"建议下界扩展: {suggestions['delta_lower']:.2%}")
print(f"建议网格数量: {suggestions['num_bins']}")
print(f"理由: {suggestions['reason']}")
```

校准数据准备

函数：

- `prepare_data_for_calibration(futures_data, option_data, model_type='cev')`

功能：

- 预处理数据用于参数校准
- 统一数据格式和清洗

状态： ☒ 已完整实现， ☒ 未被调用

使用场景：

- 批量校准时自动化数据准备
- 不同模型间的数据转换

1.5 文件工具模块 (utility/file_utils.py)

参数结果保存

函数：

- `save_parameter_results(params, param_names, model_type, result_dir='result', timestamp=True)`

功能：

- 保存参数估计结果为CSV
- 支持时间戳和多批次

状态： ☒ 已完整实现， ☒ 未被调用

使用场景：

- 保存历史参数估计结果
 - 参数趋势分析
 - 多次运行结果对比
-

模型比较保存

函数:

- `save_model_comparison(comparison_results, result_dir='result')`

功能:

- 保存模型对比结果
- 支持Excel导出

状态: ☒ 已完整实现, ☒ 未被调用

使用场景:

- CEV vs CEV-DEJD vs CEV-RS对比研究
 - 论文表格生成
 - 模型选择报告
-

期权定价结果保存

函数:

- `save_option_pricing_results(put_price, call_price, model_params, option_params, result_dir='result', timestamp=True)`

功能:

- 保存看涨/看跌期权定价结果
- 详细记录参数配置

状态: ☒ 已完整实现, ☒ 未被调用

使用场景:

- 定价结果归档
 - 详细审计追踪
-

行权边界保存

函数:

- `save_exercise_boundary_results(boundaries, bins_centers, model_type, option_params, result_dir='result', timestamp=True)`

功能:

- 保存CEV-RS早期行权边界
- 支持可视化数据导出

状态: ☒ 已完整实现, ☒ 未被调用

使用场景:

- 边界可视化数据准备
 - 边界随时间变化分析
-

机制分析结果保存

函数:

- `save_regime_analysis_results(regime_params, regime_stats, result_dir='result', timestamp=True)`

功能:

- 保存机制分析详细结果
- 包括平稳分布、持续时间等

状态: ☒ 已完整实现, ☐ 未被调用

使用场景:

- 机制风险报告
- 市场状态研究

二、预留接口但未实现的功能

2.1 Greeks计算 ⚠ 重要

函数:

- `calculate_greeks(q_matrix, bins_centers, model_params, option_params, ctmc_params, model_type='cev')`

当前状态:

- 函数框架已定义
- 返回值为占位符 (None)
- 仅打印警告信息

需要实现的Greeks:

基础Greeks (所有模型)

- Delta (Δ):** 期权价格对标的资产价格的敏感性

$$\Delta = \partial P / \partial S$$

应用: 对冲比例、风险评估

- Gamma (Γ):** Delta对标的资产价格的敏感性

$$\Gamma = \partial^2 P / \partial S^2$$

应用: Delta调整频率、凸性风险

- Theta (Θ):** 期权价格对时间流逝的敏感性

$$\Theta = \partial P / \partial t$$

应用: 时间衰减评估

- Vega (v):** 期权价格对波动率的敏感性

$$v = \partial P / \partial \sigma$$

应用：波动率风险、Vega对冲

- **Rho (ρ)**: 期权价格对无风险利率的敏感性

$$\rho = \partial P / \partial r$$

应用：利率风险

CEV-DEJD额外Greeks

- **Lambda Sensitivity**: 对跳跃强度的敏感性
- **Jump Risk**: 跳跃风险暴露

CEV-RS额外Greeks

- **Delta_Regime1**: 机制1下的Delta
- **Delta_Regime2**: 机制2下的Delta
- **Regime Sensitivity**: 对机制转换的敏感性

实现方法建议:

有限差分法 (最简单, 精度中等) :

```
def calculate_greeks_fd(q_matrix, bins_centers, model_params, option_params,
                        ctmc_params, model_type='cev', epsilon=0.01):
    """使用有限差分法计算Greeks"""

    # 基础价格
    base_price = option_pricing.price_option(q_matrix, bins_centers,
                                              model_params,
                                              option_params, ctmc_params, model_type)

    # Delta: (P(S+ε) - P(S-ε)) / (2ε)
    params_plus = copy.deepcopy(model_params)
    params_minus = copy.deepcopy(model_params)
    params_plus['F_t'] *= (1 + epsilon)
    params_minus['F_t'] *= (1 - epsilon)

    price_plus = option_pricing.price_option(q_matrix, bins_centers,
                                              params_plus,
                                              option_params, ctmc_params, model_type)

    price_minus = option_pricing.price_option(q_matrix, bins_centers,
                                              params_minus,
                                              option_params, ctmc_params,
                                              model_type)[0]

    delta = (price_plus - price_minus) / (2 * epsilon * model_params['F_t'])

    # Gamma: (P(S+ε) - 2P(S) + P(S-ε)) / ε²
    gamma = (price_plus - 2*base_price + price_minus) / (epsilon *
                                                           model_params['F_t'])**2

    # Theta: (P(t-δt) - P(t)) / δt
```

```

params_time_minus = copy.deepcopy(option_params)
params_time_minus['toT'] -= 0.001 # 小的时间步长

# ... (需要重新定价)

# Vega: 重新计算不同波动率下的价格
# ... (需要修改sigma重新定价)

# Rho: 重新计算不同利率下的价格
# ... (需要修改rf重新定价)

return {
    'delta': delta,
    'gamma': gamma,
    'theta': None, # 需要实现
    'vega': None, # 需要实现
    'rho': None, # 需要实现
}

```

注意事项:

1. 有限差分法的精度受epsilon影响
2. CEV-DEJD的Vega较复杂 (同时影响扩散和跳跃)
3. CEV-RS的Greeks需要考虑机制转换
4. 高精度方法: Adjoint法 (需要数学推导)

应用场景:

- 期权交易风险管理
- 希腊字母对冲
- 期权组合风险分析
- 灵敏度分析

2.2 隐含波动率计算

函数:

- `calculate_implied_volatility(market_price, model_params, option_params, ctmc_params, model_type='cev')`

当前状态:

- 函数框架已定义
- 返回值为None
- 仅打印警告信息

需要实现的功能:

- 给定期权市场价格, 反推隐含的波动率参数
- 类似Black-Scholes隐含波动率, 但使用CTMC定价模型

实现方法建议:

牛顿-拉夫逊法 (快速收敛):

```

def calculate_implied_volatility_nr(market_price, model_params, option_params,
    ctmc_params, model_type='cev',
    max_iter=100, tolerance=1e-6):

```

```

"""使用牛顿-拉夫逊法计算隐含波动率"""

sigma = model_params.get('sigma', 0.2) # 初始猜测
beta = model_params.get('beta', -0.5)

for i in range(max_iter):
    # 当前模型价格
    params_temp = copy.deepcopy(model_params)
    params_temp['sigma'] = sigma
    params_temp['beta'] = beta

    model_price, _ = option_pricing.price_option(
        q_matrix, bins_centers, params_temp, option_params, ctmc_params,
        model_type
    )

    # 误差
    error = model_price - market_price

    if abs(error) < tolerance:
        return sigma # 收敛

    # 数值计算导数 (Vega)
    params_plus = copy.deepcopy(params_temp)
    params_plus['sigma'] = sigma * 1.01 # 1%扰动

    # 注意: 需要重新构造Q矩阵
    price_plus, _ = option_pricing.price_option(
        q_matrix, bins_centers, params_plus, option_params, ctmc_params,
        model_type
    )

    vega = (price_plus - model_price) / (0.01 * sigma)

    # 更新猜测
    sigma = sigma - error / vega

    # 边界保护
    sigma = max(0.001, min(1.0, sigma))

raise RuntimeError(f"隐含波动率未收敛, 迭代{max_iter}次后仍不收敛")

```

注意事项:

1. CEV模型有两个参数 (σ, β) , 隐含波动率概念需要定义
 - 可选择固定 β , 求解 σ
 - 或联合优化 σ 和 β (计算量大)
2. 每次迭代需要重新构造Q矩阵 (计算成本高)
3. 可以使用更快的优化器 (Brent法等)

应用场景:

- 波动率曲面分析
- 波动率套利检测
- 市场情绪评估

附录：函数索引

模块	函数名	状态	优先级
parameter_estimation	log_likelihood_futures	已实现，未调用	中
parameter_estimation	log_likelihood_futures_rs	已实现，未调用	中
parameter_estimation	calculate_stationary_distribution	已实现，未调用	高
parameter_estimation	calculate_expected_regime_duration	已实现，未调用	高
parameter_estimation	analyze_regime_parameters	已实现，未调用	高
option_pricing	calculate_greeks	△ 占位符	最高
option_pricing	calculate_implied_volatility	△ 占位符	最高
option_pricing	monitor_pricing_convergence	已实现，未调用	低
option_pricing	analyze_option_value_profile	已实现，未调用	中
data_processor	analyze_jump_characteristics	已实现，未调用	高
data_processor	analyze_regime_characteristics	已实现，未调用	高
data_processor	analyze_leverage_effect	已实现，未调用	高
data_processor	suggest_state_space_params	已实现，未调用	中
file_utils	save_parameter_results	已实现，未调用	中
file_utils	save_model_comparison	已实现，未调用	高
file_utils	save_exercise_boundary_results	已实现，未调用	中